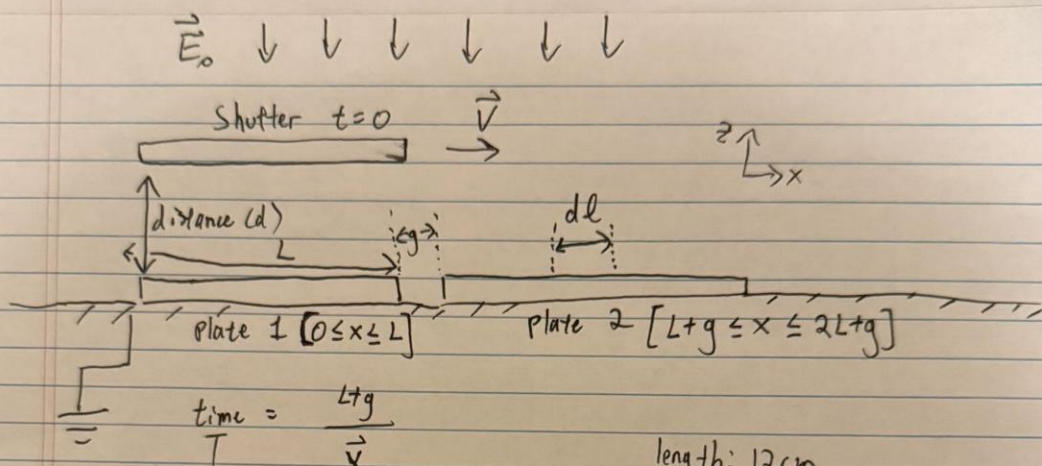


Project 1 Electrodynamics

John Li

Part 1

Hand Analysis



assumptions

- quasistatic (static field)
- Constant speed of shutter
- fringing fields is neglected.

length: 12 cm

width: 12 cm

$E_0 = 100 \text{ V/m}$

$\vec{v} = 5 \text{ m/s}$

$g = 0.5 \text{ mm} = 0.05 \text{ cm}$

$d = 1 \text{ mm} = 0.1 \text{ cm}$

$\epsilon_0 = 8.854 \times 10^{-12}$

$$\frac{d^2 V}{dz^2} = 0 \Rightarrow V(z) = C_1 z + C_2$$

below shutter $0 < z < d$, $V(0) = 0$ & $V(d) = 0$

$$\Rightarrow C_2 = 0, C_1 d = 0 \Rightarrow V(z) = 0, -\nabla V = 0$$

Exposed part

$$V(z) = E_0 z = -\hat{z} E_0$$

from Gauss law $q_{\text{surface}} = \begin{cases} -\epsilon_0 E_0 & \text{exposed} \\ 0 & \text{covered} \end{cases}$

Total charge plate

$$Q_{\text{plate}}(t) = \iint_{\text{plate 1,2}} q_{\text{surface}} ds = -\epsilon_0 \vec{E}_0 \cdot \text{Area}_{\text{plate 1,2}}(t)$$

Area vs time

Plate 1 $\Rightarrow \begin{cases} W\vec{v}t, & 0 \leq t \leq \frac{L}{\vec{v}} \quad (\text{moving}) \\ WL, & L/\vec{v} \leq t \leq T \end{cases}$

plate 2 $\Rightarrow \begin{cases} WL, & 0 \leq t \leq g/\vec{v} \quad (\text{on gap}) \\ (g+L-\vec{v}t)W, & g/\vec{v} \leq t \leq T \quad (\text{across Plate}) \end{cases}$

Current on Plate

$$\vec{i} = \frac{dq}{dt} = -\epsilon_0 \vec{E}_0 W \frac{dL}{dt}$$

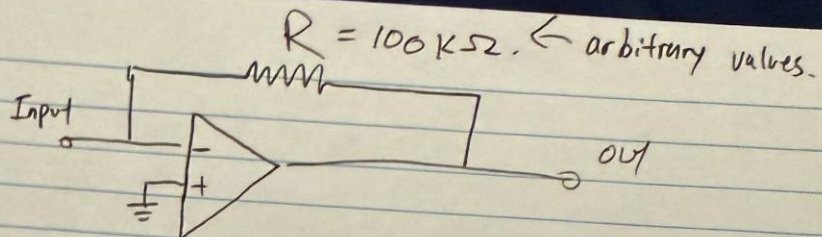
Time interval

$0 < t < g/\vec{v}$
 $g/\vec{v} < t < L/\vec{v}$
 $L/\vec{v} < t < T$

i_1
 ~~$-\epsilon_0 \epsilon_0 W \vec{v}$~~
 $-\epsilon_0 \epsilon_0 W \vec{v}$
 0

i_2
 ~~0~~
 $\epsilon_0 \epsilon_0 W \vec{v}$
 $\epsilon_0 \epsilon_0 W \vec{v}$

TIA



$$A_v = V_{out} = 2 \epsilon_0 E_0 R \vec{V}$$

Final Values

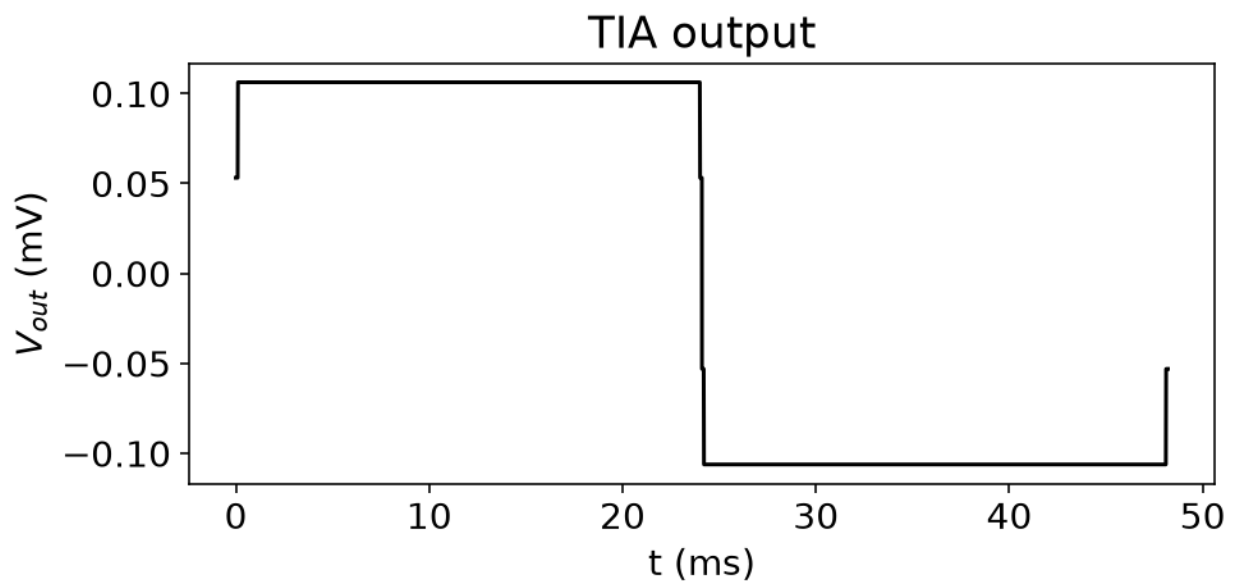
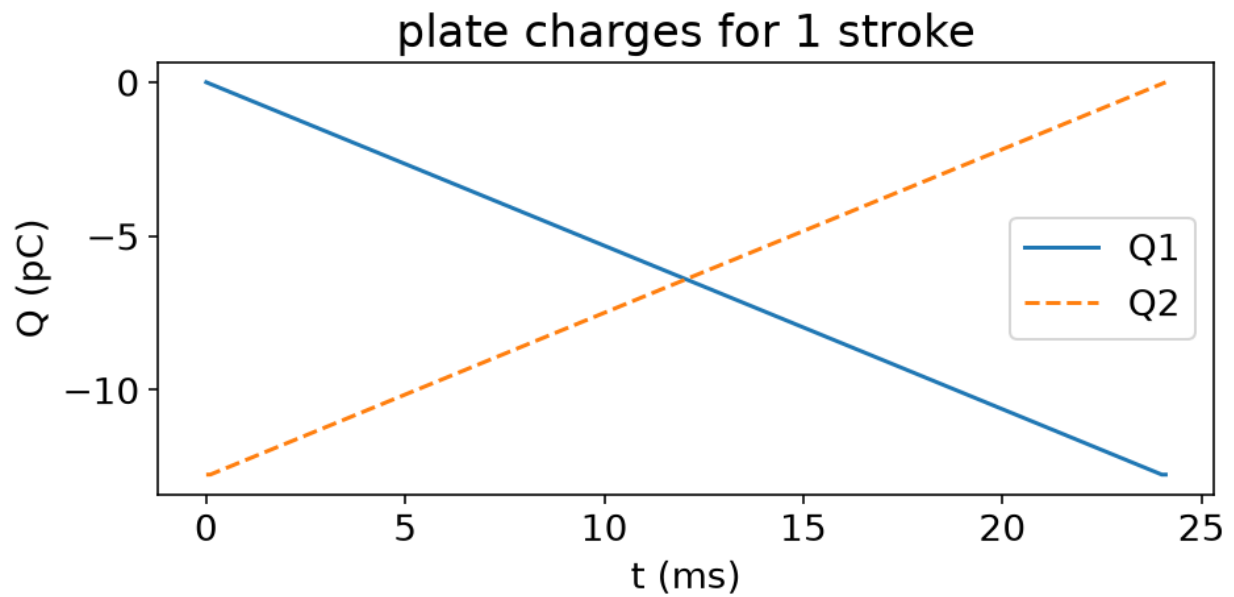
$$|q_{surface}| = 8.854 \times 10^{-10} \text{ C/m}^2$$

$$|Q_{plate}(t)| = 1.275 \times 10^{-11} \text{ C}$$

$$\cancel{I_{ph}} \dot{Q} = 5.3 \times 10^{-10} \text{ A}$$

$$V_{out} = 0.1062 \text{ mV}$$

analysis is presented in the handwritten notes below. A python program is also created to calculate the values & generate plots.



Compiler output

$q_{surface} = -8.8542e-10 \text{ C/(m}^2\text{)}$

$q_{plate} = 12.7500 \text{ pC}$

time_sweep = 0.0241 S

I_o = 0.0000000005312513 A

V_out = 0.000106250 V

images are saved after compiled output

Part 2

numerical python program screenshots is shown below. Very similar to part 1, except that the TIA waveform is more smoother than part 1, values are similar, not too far off. Code is copy pasted in last section.

Compiler output

finite difference grid: 2578 x 97 nodes (dx = 0.1000 mm near the plates, box is 1100 mm x 431 mm)

Checking (a), no shutter: Q1 = -1.275003e-11 Coulombs, expected -1.275003e-11 Coulombs

===== Sanity Checks Results =====

Plates: L = 120.000 mm, W = 120.000 mm, d = 1.000 mm, g = 0.5 mm, v = 5.0000 m/s, tia_gain = 100000.0000000000 Ohm

Stroke time T = (L+g)/v = 0.024100 seconds

Q1, plate 1 fully covered (t=0) = -0.017609 pCoulombs

Q1, plate 1 fully exposed (t=T) = -12.564453 pCoulombs

Check (b), Q1 at t = T with different grids:

base grid Q1 = -12.564453 pCoulomb change = -0.000000 %

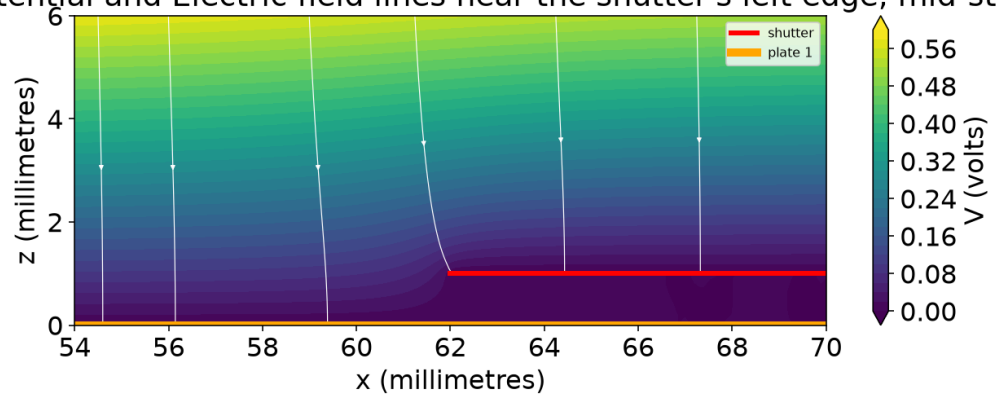
coarser grid, dx = 0.25 mm Q1 = -12.559988 pCoulomb change = -0.035537 %

larger box, 0.8 m Q1 = -12.562868 pCoulomb change = -0.012615 %

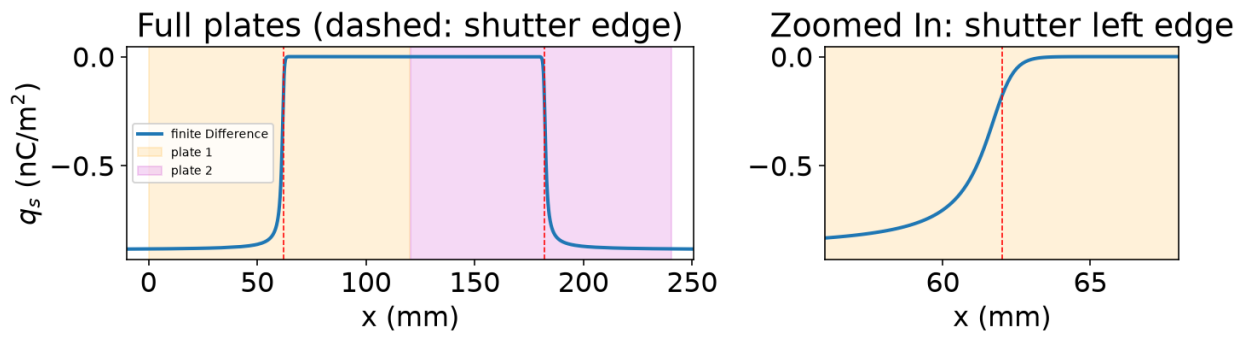
stretch = 1.08 Q1 = -12.564796 pCoulomb change = +0.002732 %

images are saved after compiled output

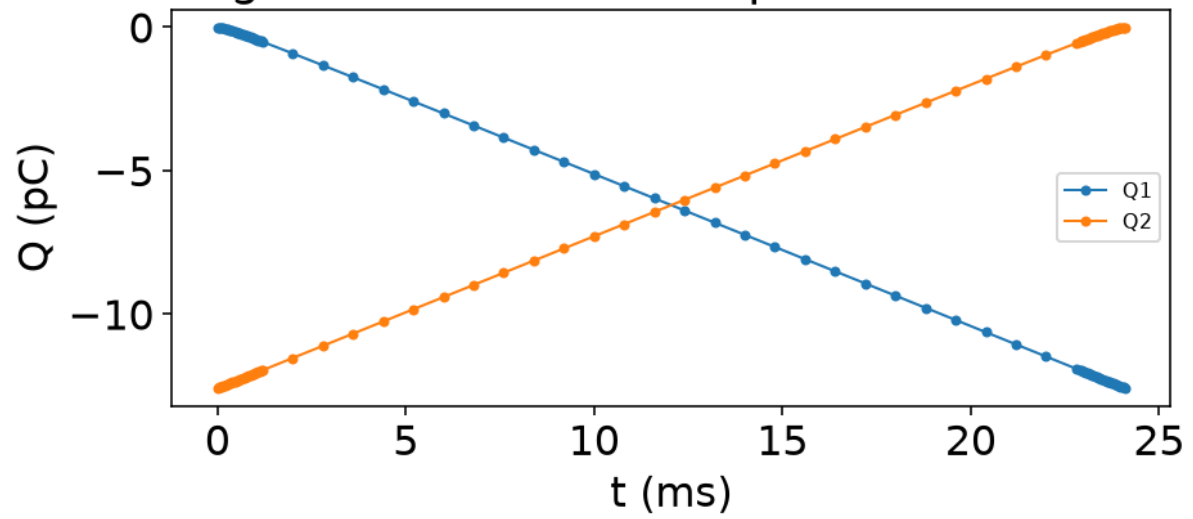
Charge Potential and Electric field lines near the shutter's left edge, mid-stroke

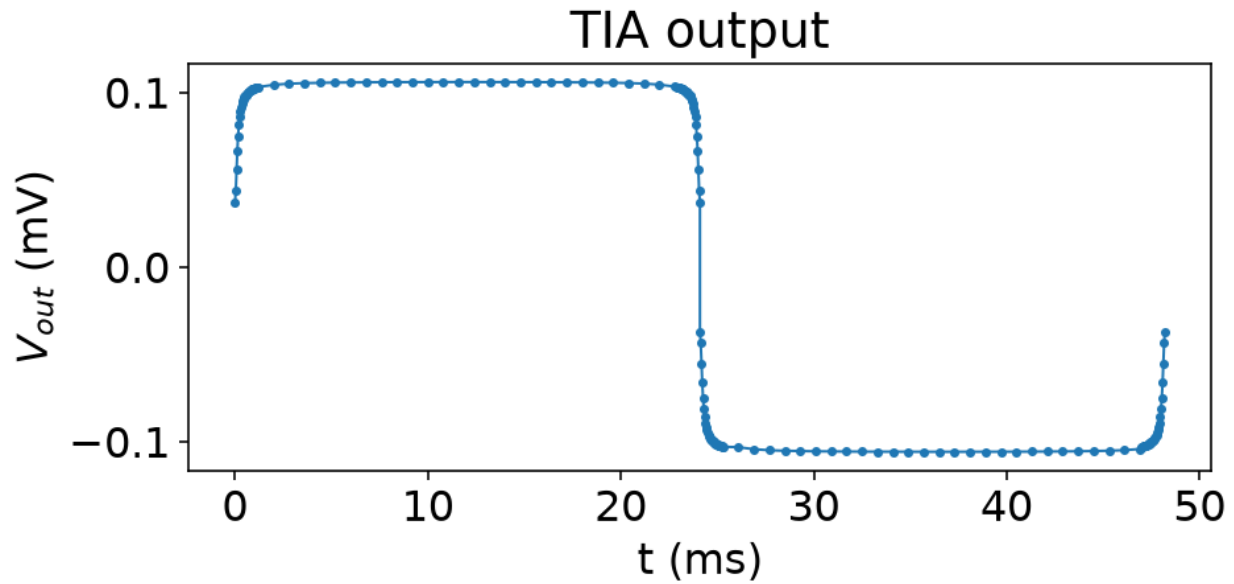


Induced surface charge q_s at 1/2 stroke



Charge Induced on sense plates in one stroke





Part 3

Numerically part 1 & 2 are similar in numbers but slight difference, since part 1 I did not assume fringe capacitance, as can be seen with the TIA output, with fringing it deviates from a square wave but has better smoothing. Overlaying the plots it would give a more clear picture. I imagine the qs of the plates would be that of a square wave when fringing is not considered.

Python Code for part 1

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
proj 1 part 1
```

```
it is what it is, i like popcorn and fried chicken & fried rice
```

```
@author: jonniboi
```

```
"""
```

```
import numpy as np
```

```
import scipy.sparse as sp
```

```
import scipy.sparse.linalg as spla
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# parameters
```

```
#L & W will be same size (12cm)
```

```
L= 12e-02
```

```
W= 12e-02
```

```
g= 0.0005 #0.5mm spacing of the sense plates
```

```
d= 1e-3 #shutter-to-sense-plate spacing
```

```
area = L*W
```

```
E_o= 100 #V/m, stated
```

```
eps_o= 8.854187817e-12
```

```
v = 5 # shutter vel
```

```
tia_gain = 100000 # TIA
```

```
time_sweep = (L + g) / v # time it takes for one stroke
```

```
q_s = -eps_o * E_o #q surface
```

```
q_plate = eps_o * E_o * area #q plate
```

```
def charges(t):
```



```
plt.rcParams.update({"font.size": 14})
```

```
fig, ax = plt.subplots(figsize=(7, 3.5))
```

```
ax.plot(t1 * 1e3, Q1 * 1e12, label="Q1") #convert to millisec
```

```
ax.plot(t1 * 1e3, Q2 * 1e12, "--", label="Q2")
```

```
ax.set_xlabel("t (ms)"); ax.set_ylabel("Q (pC)")
```

```
ax.set_title("plate charges for 1 stroke")
```

```
ax.legend(); fig.tight_layout(); fig.savefig("Tequila_part1_charge.png", dpi=150)
```

```
tP = np.concatenate([t1, time_sweep + t1])
```

```
VP = np.concatenate([V_out, -V_out[::-1]])
```

```
fig, ax = plt.subplots(figsize=(7, 3.5))
```

```
ax.plot(tP * 1e3, VP * 1e3, "k")
```

```
ax.set_xlabel("t (ms)"); ax.set_ylabel(r"$V_{out}$ (mV)")
```

```
ax.set_title(f"TIA output")
```

```
fig.tight_layout(); fig.savefig("BudLight_output.png", dpi=150)
```

```
print ("\n")
```

```
print("images are saved after compiled output")
```

Python Code for part 2

```
# -*- coding: utf-8 -*-
```

```
"""
```

proj 1 part 2

I used spyder w/ mods

it is what it is, i like popcorn and fried chicken & fried rice

@author: jonniboi

```
"""
```

```
import numpy as np
```

```
import scipy.sparse as sp
```

```
import scipy.sparse.linalg as spla
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
#paramters -----
```

#L & W will be same size (12cm)

L= 12e-02

W = 12e-02

g= 0.0005 #0.5mm spacing of the sense plates

d= 1e-3 #shutter-to-sense-plate spacing

*area = L*W*

E_o= 100 #V/m, static electric field

eps_o= 8.854187817e-12 #vacuum permittivity F/m

z_core= 0.006 # z region

*core_pad= 0.005 # distance the grid extends past
#the outer ends of the two sense plates*

X_ps= 0.4 #distance form plate to side wall

stretch= 1.2 #ratio of cell size outside fine grid region

tia_gain= 1e5 #TIA gain

v= 5 #linear shutter velocity m/s

dx= 0.0001 #grid spacing

pulse_segments= 2000

time_sweep= (L+g)/v #time it takes for one stroke

plate1= (0,L) #plate 1

*plate2= (L+g, L*2+g)*

X_far = 0.4 # meters, distance from the plates to side-walls

H = 0.4 # meters, ~height box region

#-----

def stretched(h0, r, length):

```
pts, i, h = [] , 0, h0
```

```
while i < length:
```

```
    h *= r
```

```
    i += h
```

```
    pts.append(i)
```

```
return np.array(pts)
```

```
def grid_space(dx, X_ps, H, stretch):
```

```
    n_L = int(round(L/dx)) #plate divided by shutter length L
```

```
    n_g = int(round(g/dx)) #converts to integer for the grid and array positions
```

```
    n_d = int(round(d/dx))
```

```
    n_c0 = int(round (core_pad/dx))
```

```
    n_zc= int(round(z_core/dx))
```

```
    x_1=np.arange(-n_c0,2*n_L+n_g+n_c0+1)*dx
```

```
    x_2= stretched(dx, stretch, X_ps)
```

```
x= np.concatenate([x_1[0]-x_2[:-1], x_1, x_1[-1] + x_2])
```

```
z= np.concatenate ([np.arange(n_zc+1)*dx,
```

```
                    n_zc*dx+stretched(dx, stretch,
```

```
                    H-n_zc*dx]))
```

```
return dict(x=x, z=z, dx=dx,
```

```
          n_L=n_L, n_d=n_d, n_zc=n_zc,
```

```
          n_m=len(x_2)+n_c0,
```

```
          ic=slice(len(x_2), len(x_2)+len(x_1)))
```

```
def laplace_eq_solve(G,x_s): #solve laplace
```

```
    x, z = G["x"], G["z"]
```

```
    nx, nz = len(x), len(z)
```

```
    N= nx*nz
```

```
idx = np.arange(N).reshape(nz,nx)
```

```
#boundary stuffs
```

```
fixed = np.zeros((nz, nx), bool)
```

```
val = np.zeros((nz, nx))
```

```
fixed[0, :] = True
```

```
fixed[-1, :] = True
```

```
val[-1, :] = E_o * z[-1]
```

```
if x_s is not None: # shutter @ z=d, V = 0
```

```
    i0 = G["n_m"] + int(round(x_s / G["dx"]))
```

```
    fixed[G["n_d"], i0:i0 + G["n_L"] + 1] = True
```

```
# nx × nz nodes
```

```
K, I = np.nonzero(~fixed)
```

```
P = idx[K, I]
```

```
Iw = np.where(I == 0, 1, I - 1)
```

```
Ie = np.where(I == nx - 1, nx - 2, I + 1)
```

```
## x[I] x-position node
```

```
hw = np.abs(x[I] - x[Iw]); he = np.abs(x[Ie] - x[I])
```

```
hs = z[K] - z[K - 1]; hn = z[K + 1] - z[K]
```

```
#finite difference
```

```
cW = 2 / (hw * (hw + he)); cE = 2 / (he * (hw + he))
```

```
cS = 2 / (hs * (hs + hn)); cN = 2 / (hn * (hs + hn))
```

```
cP = -(cW + cE + cS + cN)
```

```
#store the equations in matrix form
```

```
rows = np.concatenate([P, P, P, P, P])
```

```
cols = np.concatenate([P, idx[K, Iw], idx[K, Ie], idx[K - 1, I], idx[K + 1, I]])
data = np.concatenate([cP, cW, cE, cS, cN])
```

```
Kf, If = np.nonzero(fixed)
Pf = idx[Kf, If]
```

```
#equations for fixed nodes
rows = np.concatenate([rows, Pf])
cols = np.concatenate([cols, Pf])
data = np.concatenate([data, np.ones(len(Pf))])
```

```
#2-D boundary-value array
Amat = sp.csc_matrix((data, (rows, cols)), shape=(N, N))
return spla.spsolve(Amat, val.ravel()).reshape(nz, nx)
```

```
def q_s(G,V): #surface charge
```

```
dV_dz= (-3*V[0]+4*V[1]-V[2])/(2*G["dx"])
return -1*eps_o * dV_dz
```

```
def q_c(G, q_es, range): #plate charge
```

```
x = G["x"]
m = (x >= range[0] - 1e-12) & (x <= range[1] + 1e-12)
return W * np.trapezoid(q_es[m], x[m])
```

```
def q_c_shutter(G, x_s): #chqrge when shuttering
```

```
V = laplace_eq_solve(G, x_s)
```

$q = q_s(G, V)$

$\text{return } q_c(G, q, \text{plate1}), q_c(G, q, \text{plate2}), V, q$

#-----

$G = \text{grid_space}(dx, X_far, H, \text{stretch})$

$x, z = G["x"], G["z"]$

$\text{print}(f\text{"finite difference grid: \{len(x)\} x \{len(z)\} nodes (dx = \{dx*1e3:.4f\} mm near the plates, "$
 $f\text{"box is \{(x[-1]-x[0])*1e3:.0f\} mm x \{z[-1]*1e3:.0f\} mm})"$

Checking purposes

$Q_free = q_c_shutter(G, \text{None})[0]$

$\text{print}(f\text{"Checking (a), no shutter: } Q1 = \{Q_free:.6e\} \text{ Coulombs, expected } \{-eps_o*E_o*area:.6e\}$
 $\text{Coulombs}")$

$\text{stroke} = L + g$

$x_pos = \text{np.unique}(\text{np.round}(\text{np.concatenate}([$
 $\text{np.arange}(0, 6.0001e-3, 0.2e-3),$
 $\text{np.arange}(6e-3, \text{stroke} - 6e-3, 4e-3),$
 $\text{np.arange}(\text{stroke} - 6e-3, \text{stroke} + 1e-9, 0.2e-3), [\text{stroke}]))/dx) * dx)$

$Q1, Q2 = [], []$ #plate1 & plate 2

for k, xs in $\text{enumerate}(x_pos)$: #position of shutter for the current pass of loop

$q1, q2, V, q = q_c_shutter(G, xs)$

$Q1.append(q1); Q2.append(q2)$

if $k == \text{len}(x_pos) // 2$:

$V_mid, q_mid, x_sh_mid = V, q, xs$

$Q1, Q2 = \text{np.array}(Q1), \text{np.array}(Q2)$

$t1 = x_pos/v$ # time for each position

```
#-----
```

```
#output currents and voltages
```

```
i1 = np.gradient(Q1, t1)
```

```
i2 = np.gradient(Q2, t1)
```

```
V_out = tia_gain * (i2 - i1) #TIA
```

```
# avg abs V_out / one stroke = TIA gain * (charges that are removed) / time_sweep
```

```
V_avg = tia_gain * (abs(Q1[-1] - Q1[0]) + abs(Q2[-1] - Q2[0])) / time_sweep
```

```
#-----
```

```
#-----
```

```
#output in COMPILER
```

```
print(f"\n===== Sanity Checks Results =====")
```

```
print(f"Plates: L = {L* 1e3:.3f} mm, W = {W*1e3:.3f} mm, d = {d* 1e3:.3f} mm, "
```

```
      f"g = {g*1e3:.1f} mm, v = {v:.4f} m/s, tia_gain = {tia_gain:.9f}Ohm")
```

```
print(f"Stroke time T = (L+g)/v = {time_sweep:.6f} seconds")
```

```
print(f"Q1, plate 1 fully covered (t=0) = {Q1[0]*1e12:.6f} picoCoulombs")
```

```
print(f"Q1, plate 1 fully exposed (t=T) = {Q1[-1]*1e12:.6f} picoCoulombs")
```

$$xc, zc = x[ic], z[:n - zc + 1]$$

```

Vc = V_mid[:n_zc + 1, ic]
X, Zg = np.meshgrid(xc_ * 1e3, zc_ * 1e3)
cs = ax.contourf(X, Zg, Vc, levels=np.linspace(0, E_o * 6e-3, 31),
                 cmap="viridis", extend="both")

Ez, Ex = np.gradient(-Vc, dx)
under = (Zg < d * 1e3) & (X > x_sh_mid * 1e3) & (X < (x_sh_mid + L) * 1e3)
Ex = np.ma.masked_where(under, Ex); Ez = np.ma.masked_where(under, Ez)
seeds = np.column_stack([np.arange(x_sh_mid * 1e3 - 7.5, x_sh_mid * 1e3 + 8, 0.75),
                          np.full(21, 5.9)])
ax.streamplot(xc_ * 1e3, zc_ * 1e3, Ex, Ez, color="w", linewidth=0.6,
              arrowsize=0.6, start_points=seeds, density=3)

ax.plot([x_sh_mid * 1e3, (x_sh_mid + L) * 1e3], [d * 1e3] * 2, "r", lw=3, label="shutter")
ax.plot([plate1[0] * 1e3, plate1[1] * 1e3], [0, 0], "orange", lw=5, label="plate 1")
ax.set_xlim(x_sh_mid * 1e3 - 8, x_sh_mid * 1e3 + 8); ax.set_ylim(0, 6)
ax.set_xlabel("x (millimetres)"); ax.set_ylabel("z (millimetres)")
ax.set_title("Charge Potential and Electric field lines near the shutter's left edge, mid-stroke")
fig.colorbar(cs, label="V (volts)"); ax.legend(loc="upper right", fontsize=8)
fig.tight_layout(); fig.savefig("I_am_the_storm_that_is_approaching.png", dpi=150)

# Fig 2: q_s on z = 0 (middle of the stroke)
fig, axs = plt.subplots(1, 2, figsize=(10, 3.6), gridspec_kw={"width_ratios": [1.6, 1]})
for ax, xl in zip(axs, [(-10, 2 * L * 1e3 + g * 1e3 + 10),
                        (x_sh_mid * 1e3 - 6, x_sh_mid * 1e3 + 6)]):
    ax.plot(x * 1e3, q_mid * 1e9, "C0", lw=2, label="finite Difference")
    ax.axvspan(plate1[0] * 1e3, plate1[1] * 1e3, color="orange", alpha=0.15, label="plate 1")
    ax.axvspan(plate2[0] * 1e3, plate2[1] * 1e3, color="m", alpha=0.15, label="plate 2")
    ax.axvline(x_sh_mid * 1e3, color="r", lw=0.8, ls="--")

```

```

    ax.axvline((x_sh_mid + L) * 1e3, color="r", lw=0.8, ls="--")
    ax.set_xlim(*xl); ax.set_xlabel("x (mm)")

    axs[0].set_ylabel(r"$q_{s}$ (nC/m$^2$)")
    axs[0].set_title("Full plates (dashed: shutter edge)")
    axs[1].set_title("Zoomed In: shutter left edge")
    axs[0].legend(fontsize=7, loc="center left")
    fig.suptitle(r"Induced surface charge $q_{s}$ at 1/2 stroke")
    fig.tight_layout(); fig.savefig("Yippy.png", dpi=150)

# Fig 3: q_c vs time plot
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(t1 * 1e3, Q1 * 1e12, "C0o-", ms=3, lw=1, label="Q1")
ax.plot(t1 * 1e3, Q2 * 1e12, "C1o-", ms=3, lw=1, label="Q2")
ax.set_xlabel("t (ms)"); ax.set_ylabel("Q (pC)")
ax.set_title("Charge Induced on sense plates in one stroke")
ax.legend(fontsize=8); fig.tight_layout(); fig.savefig("Chilling.png", dpi=150)

# TIA output over one cycle
tP = np.concatenate([t1, time_sweep + t1])
VP = np.concatenate([V_out, -V_out[::-1]])
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(tP * 1e3, VP * 1e3, "C0o-", ms=2.5, lw=1)
ax.set_xlabel("t (ms)"); ax.set_ylabel(r"$V_{out}$ (mV)")
ax.set_title(f"TIA output")
fig.tight_layout(); fig.savefig("Stuffs.png", dpi=150)

print("images are saved after compiled output")

```

